2025 IEEE International Symposium on High-Performance Computer Architecture (HPCA)

# Enhancing Large-Scale AI Training Efficiency: The C4 Solution for Real-Time Anomaly Detection and Communication Optimization

Jianbo Dong†, Bin Luo†, Jun Zhang†, Pengcheng Zhang†, Fei Feng†, Yikai Zhu†, Ang Liu†, Zian Chen†, Yi Shi†, Hairong Jiao†, Gang Lu†, Yu Guan†, Ennan Zhai†, Wencong Xiao†, Hanyu Zhao†, Man Yuan†, Siran Yang†, Xiang Li†, Jiamang Wang†, Rui Men†, Jianwei Zhang†, Chang Zhou†, Dennis Cai†, Yuan Xie†‡, Binzhang Fu†
† Alibaba Group ‡ Hong Kong University of Science and Technology
jianbo.djb@alibaba-inc.com

arXiv:2406.04594v2 [cs.DC] 23 May 2025

***Abstract*—The emergence of Large Language Models (LLMs) has necessitated the adoption of distributed training techniques, involving the deployment of thousands of GPUs to train a single model. Unfortunately, the efficiency of large-scale distributed training systems is often suboptimal due to the increased likelihood of hardware errors in high-end GPU products and the heightened risk of network traffic collisions. Specifically, GPUs involved in the same job require periodic synchronization to exchange necessary data, such as gradients, parameters, or activations. As a result, any local hardware failure can disrupt training tasks, and the inability to swiftly identify faulty components leads to a significant waste of GPU resources. Moreover, prolonged communication due to traffic collisions can substantially increase GPU waiting times.**

**To address these challenges, we propose a communication-driven solution, namely the C4. The key insights of C4 are twofold. First, the load in distributed training exhibits homogeneous characteristics and is divided into iterations through periodic synchronization, therefore hardware anomalies would incur certain syndrome in collective communication. By leveraging this feature, C4 can rapidly identify the faulty components, swiftly isolate the anomaly, and restart the task, thereby avoiding resource wastage caused by delays in anomaly detection. Second, the predictable communication model of collective communication, involving a limited number of long-lived flows, allows C4 to efficiently execute traffic planning, substantially reducing bandwidth competition among these flows. The C4 has been extensively deployed across real-world production systems in a hyperscale cloud provider, yielding a significant improvement in system efficiency, from 30% to 45%. This enhancement is attributed to a 30% reduction in error-induced overhead and a 15% reduction in communication costs.**

## I. Introduction

The rapid advancement of Large Language Models (LLMs) [43], [44], [5], [51], [15], [2] has significantly impacted the fields of machine learning (ML) and artificial intelligence (AI). LLMs such as GPT [43] and Llama [51] families are exhibiting human-like proficiency in text generation, question-answering, and even code creation. Despite the promising capabilities of these models, training them to reach their full potential is a complicated and resource-intensive process. Training a GPT-like model that has 175 billion parameters, for example, may require up to two months in a cluster of 1000 GPUs [33].

One of the major hurdles we faced when training LLMs is the suboptimal utilization of GPU resources, which can be attributed to two primary reasons. First, a significant amount of time (over 30%) is spent on recovering from system errors throughout the training lifespan. Extended training sessions necessitate a stable process; however, the latest generation GPUs [35], [37] tend to exhibit high error rates, likely a consequence of their rapid development, rushed delivery, and increased power consumption. When running in Bulk Synchronous Parallel (BSP) [52] mode, errors in any node can cause the entire job to fail. This necessitates extensive time for system diagnosis and node isolation before we restart the job.

Furthermore, tuning the training process of these models to peak efficiency is a complex task, with communication costs being a significant impediment to scalability. That is, GPUs may experience delays at synchronization points while awaiting the outcomes of collective operations. The state of the arts [13], [14] suggest managing network traffic to improve communication performance. However, a shared physical cluster with multiple tenants running concurrent training jobs adds to the complexity of these patterns, exacerbating the difficulty in effective management. Additionally, the appearance of downlinks, which is increasingly common in large-scales systems, also diminishes the efficacy of previous traffic engineering strategies.

In order to tackle these challenges, this paper introduces Calibrating Collective Communication over Converged Ethernet (C4), an innovative system designed to unleash the computational capabilities of massive collaborating GPUs in large-scale AI clusters, by harnessing the inherent characteristics of distributed training. Specifically, it minimizes downtime caused by hardware errors by enabling the design of a highly efficient error recovery mechanism. Leveraging the homogeneous running rhythm of the distributed workers, it enables real-time fault detection by analyzing the timing discrepancies among them at the synchronization points,

namely, the collective communication operations in each step. Moreover, C4 improves runtime communication efficiency through global traffic engineering. By exploiting the features of network traffic in AI clusters, C4 implements a cluster-level path management mechanism, mitigating the impact of traffic collisions and ensuring seamless collaboration among GPUs.

The contributions of this paper include:

- We extend the capabilities of Alibaba Collective Communication Library (ACCL) [14] to support real-time status monitoring and communication path control, enabling the development of real-time error detection and global traffic engineering.
- We design and implement a real-time hardware error detection subsystem that collects and analyzes the runtime status of ACCL across different workers, and builds upon this a fast recovery system that automatically isolates faulty nodes and restarts the affected jobs.
- We develop a global traffic engineering subsystem that manages all the networking path resources in a cluster, and intelligently controls path allocation for establishing connections in ACCL, ensuring balanced load distribution across available paths.
- We successfully deploy C4 in our AI clusters, serving key customers for LLM training. It effectively removes the majority of error-induced overhead, typically accounting for about 30% of the total time. Additionally, C4 mitigates the communication costs, and improves the system throughput by approximately 15%.

## II. Understanding the Challenges in Operational AI Clusters

### A. New Challenges

In response to the immense success of LLMs, cloud providers have deployed dedicated AI clusters to effectively manage the growing demand for large-scale training tasks. Unlike traditional data centers for traditional cloud applications, large-scale AI clusters face distinct challenges arising from emerging workloads and rapidly evolving hardware systems.

*One of the key challenges is that distributed training jobs are highly sensitive to local failures, which occur with notable frequency in modern large-scale training systems.* This sensitivity arises from the periodical synchronization required among workers in each training step. It is well-known that contemporary LLMs are capable of harnessing tens of thousands of GPUs for their distributed training processes. The mainstream distributed training frameworks [49], [45] support a variety of parallel training strategies across multiple dimensions. With these parallel training strategies, each GPU manages a fraction of the model or input for localized computations. The workers within the same parallelism group must exchange gradients, parameters, or activations for further processing. If an exception occurs in any worker, it will block subsequent workers and cause the entire job to crash. Consequently, distributed training jobs, similar to HPC applications, must frequently save checkpoints and restart the entire job from the last valid checkpoint in the event of errors. What's worse, our experience has shown that high-end GPU products tend to have higher error rates, increasing the probability a distributed training job may crash, especially as the system scales out. The recovery from job crashes can be costly and time-consuming, leading to a significant waste of computational capacity.

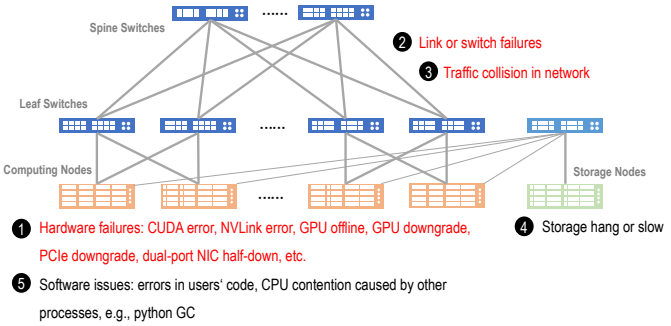



Fig. 1: Issues that can adversely affect system utilization.

Another key challenge comes from the pronounced disparity in expected delivery standards of AI clusters. Specifically, users demand peak task performance, driven by two primary motivations. First, the value of the developed models is intrinsically tied to how swiftly they can be brought to market, making expedited training a priority. Second, the significant capital invested in the training infrastructure means that any performance degradation translates directly into substantial financial repercussions. *However, maintaining a consistently optimal operational state for these large-scale systems is particularly challenging over extended periods, especially as the number of GPUs in a single job reaches thousands or even tens of thousands.*

### B. Pain Points and Their Influences on Training Jobs

Fig. 1 illustrates the various issues that can negatively impact system utilization, including ❶ anomalies in computing nodes, ❷ failures and ❸ traffic collision in networking subsystem, ❹ I/O hang or slow in storage subsystem, and ❺ problems in users' code or supporting software. First of all, nodes with critical failures may cause job crashes. And some of the computing nodes could exhibit low performance, commonly referred to as slow nodes, often due to defects in GPUs or NICs. The likelihood of encountering defect nodes increases with system scales, ultimately undermining the efficiency of the entire system. Furthermore, traffic collisions among the workers, whether from the same or different jobs, can prolong the communication process and subsequently reduce system throughput. Link and switch failures in network subsystem could exacerbate the extent of traffic collision. In addition, issues in storage subsystem and software layer would systematically affect all the workers.

From users' perspective, system utilization is impaired in two distinct ways, as illustrated in Fig. 2. First, critical failures cause frequent job crashes, leading to a waste of computing power. Failures encountered during system startup necessitate

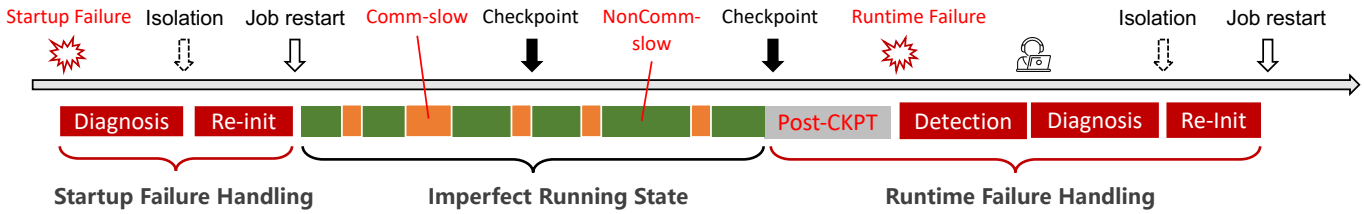



Fig. 2: The influences of system issues on training jobs: 1) job failures (at startup and runtime) incurred overhead, and 2) slowdowns in communication or non-communication processes (computing or data loading).

additional time for system diagnosis, error component isolation, and job restart. While, if failures occur at runtime, there are additional overheads including post-checkpoint and fault detection costs. Upon crashing, the job must revert to the last valid checkpoint, rendering the execution time after the last checkpoint useless, which we refer to as post-checkpoint cost. There is also a delay between the occurrence of an error and the users' awareness, known as the fault detection cost. Besides job crashes, non-critical issues at runtime, such as performance degradation in GPUs or PCIe fabric, and network traffic collisions, can also adversely affect system throughput. The extended communication time is referred to as communication slowdown, while the prolonged computing or data loading time is referred to as non-communication slowdown. Any overhead in communication or non-communication indicates degradation in system throughput. We would discuss the causes and effects of job crashes and runtime slowdowns respectively.

### C. The Quantitative Analyses of Job Crashes.

To investigate the underlying causes of job crashes and how the system utilization is impacted, we chose a representative job (one of jobs with largest scale) for analysis. This particular job, running in a newly deployed cluster, witnessed a total of 40 crashes within one month. We categories these crash causes and show their distribution in Tab. I.

*The table reveals that it's challenging to discern the underlying causes from the user's perspective, as the majority of them simply show themselves as "NCCL Errors".* This is largely due to the synchronous nature of distributed training tasks. For instance, a hardware issue in a GPU could lead to the crash of one of the worker processes, causing its peer workers to fail to receive an acknowledgment after sending a message to the crashed worker. Consequently, this triggers an error code of 12 in the communication library. This same error code could also be generated by other issues such as network disconnections, memory overflows resulting in container eviction, or users' code errors like tensor size mismatches. Regrettably, the industry lacks efficient tools to identify the root causes of these errors in a timely manner.

*This table also reveals an important fact: the majority of errors (approximately 82.5%) are confined to specific nodes or individual devices, largely attributed to the elevated error rates in high-end GPU productions.* This indicates the potential for isolating the localized faulty components, presenting an opportunity to tackle the majority of errors[1]. Nevertheless, due to the similarity in symptoms for most failures, manually identifying the root causes and locating the defective nodes could consume hours to days, particularly in a cluster with thousands of GPUs.

TABLE I: The distribution of crash causes recorded over a one-month from a representative job employing 4,096 GPUs.

| Users' View | Root Causes | Proportion | Local |
|---|---|---|---|
| NCCL Error | CUDA Error | 12.5% | 100% |
| NCCL Error | ECC/NVLink Error | 27.5% | 100% |
| NCCL Error | NCCL timeout | 20% | 75% |
| NCCL Error | ACK timeout | 27.5% | 81.8% |
| Network Error | Others | 12.5% | 40% |

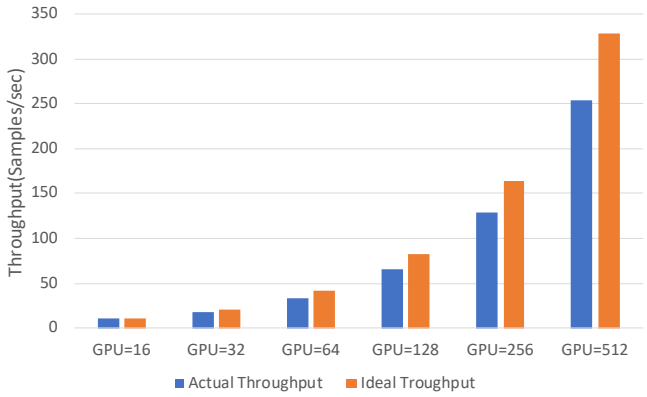



Fig. 3: Performance loss grows with system scale, due to the increased extent of traffic collision.

In our experience, around 30% of our time was dedicated to error detection, system diagnosis, isolating defective nodes, and restarting processes, leaving only 70% for actual computing tasks, in the absence of efficient fault detection and diagnostic tools. To minimize the failure-incurred costs, it is essential to address each contributing factor appropriately. For swift diagnosis, our system automatically identifies defective nodes by leveraging the inherent characteristics of training jobs, such as balanced computing costs within a data parallel group and synchronization between iterations. Upon locating a defect, the system promptly initiates isolation and restart procedures, deferring in-depth root cause analysis to offline processing. By developing automatic fault detection tool, we can eliminated human intervention, expediting the process from tens of minutes to mere seconds. In addition, to curtail the

[1]Other systematic faults, such as faults in distributed storage subsystem, would be handled by dedicated monitoring tools

post-checkpoint cost, we have implemented a rapid checkpoint mechanism, similar to the prior work [53], capable of saving checkpoints approximately every 10 iterations.

### D. The Quantitative Analyses of Runtime Slowdowns.

Since non-critical failures in individual nodes can be addressed using the same strategy as critical errors, we will put emphasis on communication slowdowns in network. To begin with, we designed a specialized high-performance network architecture for AI clusters, with each computing node equipped with multiple high-performance Network Interface Cards (NICs) supporting Remote Direct Memory Access (RDMA) to facilitate fast inter-node communication among the heavily packed GPUs in each node. Each NIC provides two physical ports, connected to a pair of leaf switches, which are interconnected to spine switches through the Fat-Tree topology, with a 1:1 oversubscription ratio. This architecture offers distinct advantages: It provides high system availability and maintainability, allowing users to react more flexibly to non-critical link failures, such as scheduling timely restarts or initiating on-demand checkpoints that are strategically based on recent checkpointing activities. Additionally, it expands the maximum system scale by doubling the number of spine switches in the system, consequently doubling the total computational capacity of GPUs.

Just as a coin has two sides, this architecture also presents challenges in communication performance. *It's known that elevated communication costs may arise from traffic collision among various flows.* When numerous flows contend for the limited bandwidth of a single link, the flow completion time can increase by several times. In collective communication, any flow that is throttled can have a ripple effect, hindering the entire communication group from proceeding with subsequent operations. *Moreover, a new problem arises with dual-port NICs. If there is an imbalance in the loads on the two physical ports, the port with the higher load can become a bottleneck.* For today's distributed training jobs, these issues can significantly prolong the communication latency and lead to degradation of communication performance. As modern models and the training clusters grow in scale, this problem would be further exacerbated. Fig. 3 presents a comparison of actual versus ideal performance, training a representative GPT-like model composed of 22 billion parameters. It is evident that there exists a discrepancy between the effective performance achieved and the theoretical ideal performance, and this disparity amplifies as the scale of the system increases. Notably, when the system scales to 512 GPUs, the effective performance drops to 30% below the ideal performance.

*Our analysis reveals that communication patterns in AI clusters markedly differ from those in traditional cloud environments.* In conventional cloud settings, we commonly observe a large number of concurrent connections, often several tens of thousands per instance. Such a volume of connections can be distributed across the network with relatively low variation using the Equal-Cost Multi-Path (ECMP) [19] routing strategy. Conversely, in distributed training systems, there are only a few longer-lasting connections, with each node typically managing around a few hundred connections. Traffic collisions in this environment can lead to substantial variations in the effective bandwidth of these connections, resulting in multi-fold increases in communication delays. However, this challenge is accompanied by an opportunity: the limited number of connections and their predictable patterns in AI clusters offer a unique advantage for enhancing communication efficiency through comprehensive global traffic engineering.

In a nutshell, the effective performance of large-scale training clusters is significantly influenced by hardware defects and traffic collisions. To maximize GPU utilization, it is crucial to reduce the time spent on fault detection and system diagnosis, thereby minimizing downtime. Additionally, eliminating unnecessary communication overhead is vital to prevent GPU stalls during periodic synchronization, ensuring optimal runtime performance.

## III. Design

To tackle the key issues identified earlier, i.e., the overheads induced by local failures and unnecessary slowdowns in communication and other processes. We will explain the rationale behind our choice of system architecture, the compromises we have accepted, and offer an in-depth analysis of the particular technical hurdles we encountered.

### A. Mitigating Error-induced Downtime

As demonstrated in the previous section, the significant hardware instability in computing nodes is a primary factor contributing to the inefficiency of AI clusters. To mitigate the impact of hardware failures, we have adopted a two-fold approach: 1) reducing hardware failure rates and 2) systematically tolerating such failures. *While the first aspect is beyond the scope of this paper, we briefly mention the importance of temperature control.* Techniques such as Dynamic Voltage and Frequency Scaling (DVFS) have been utilized to prevent GPU overheating, although they can impact performance consistency. We have enhanced cooling through increased fan speed and improved air conditioning, allowing for temperature regulation with minimal cost impact. The industry is also moving towards more efficient cooling solutions such as liquid cooling for future AI infrastructures.

With respect to the second aspect, we have collaborated with relevant teams to explore multiple solutions. Traditional cloud applications typically utilize online fault tolerance techniques. These may involve withstanding computational faults through redundant computations [41], [32], [50], ensuring reliable storage with erasure coding (EC) [55], [56], [11], [1] and/or triple-replica techniques, and mitigating network anomalies with multi-path (dual uplinks) strategies [18], [29]. For high-performance computing applications, the prevailing approach involves offline fault tolerance methods [24]. This includes the periodic saving of checkpoints, allowing tasks to be restarted from the last checkpoint in the event of system failure. Yet, any computations made between the last checkpoint and the time of the failure will be lost. In our large-scale parallel training

systems, we have implemented a practical and hybrid technical strategy. This includes adopting proven online fault-tolerant technologies in distributed storage and networking systems. While for computing resources, it's more cost-effective to use backup resources and offline fault-tolerant techniques, given the pricing of GPU products. As a result, we have allocated 64 backup GPUs across 8 servers for every 1024 GPUs on 128 servers, ensuring consistent communication and performance for parallel training on any of the 128 servers within this 136-server pool.

**Our approach: C4D.** The system that we designed to handle computational faults is shown in Fig. 4. The core of this system is the C4D (C4 Diagnose) subsystem, designed to quickly detect hardware failures for prompt task restarts. Initially, C4 agents monitor the operational status of training workers and transmit the data to a centralized master. The master then evaluates the well-being of the training workers by comparing the gathered information. If any irregularities are detected, it informs the job steering service to isolate the problematic nodes and restart the job from the most recent valid checkpoint. Simultaneously, it also sends the events to a background root cause analysis system for offline diagnosis.

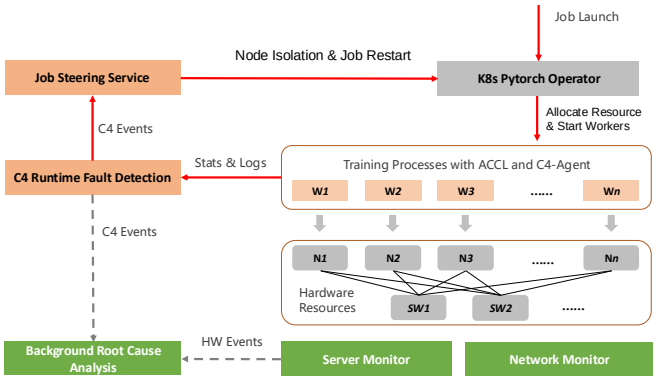



Fig. 4: The working flow of C4D.

The effectiveness of C4D is grounded in two key insights: (1) parallel training tasks exhibit a regular and predictable running rhythm, enabling us to precisely identify abnormal behaviors; (2) the Bulk Synchronous Parallel (BSP) execution model in parallel training requires regular synchronization points. And these synchronization points are used as anchors for measuring anomalies. Building on these insights, we developed the C4D to facilitate online fault detection, as shown in Fig. 5. The key elements of the C4D includes the enhanced communication library ACCL, a central C4D master, and the C4a (C4 agent) that acts as an intermediary. Here, ACCL is enhanced to provide the capability for online monitoring of collective communication operations.

**C4D Monitoring.** Fig. 6 shows the architecture and enhancement in ACCL. We have extended the monitoring capabilities of the bottom three layers, namely the communicator, operation, and transport layers, in a top-down fashion. We gather information on communicators, encompassing communicator IDs, involved devices, and device ranks. Additionally,

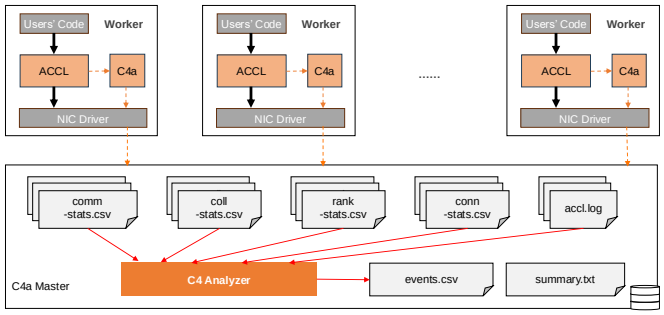



Fig. 5: The architecture of C4D.

we record details about collective operations, such as operation types, communication algorithms, data types, element counts, and duration. Furthermore, we track the startup and completion of specific collective operations and assign each operation a sequence. Lastly, we collect data in the transport layer, including connection information (e.g., source and destination IPs, QP numbers, and source ports), as well as message details like message counts, sizes, and transfer duration. Notably, it's not trivial to collect above information at runtime, with low cost and high accuracy. To precisely monitor communication kernel execution patterns, we refined all related CDUA kernels to log their start and completion times directly, as CPU timestamps and CUDA events are ineffective or inaccurate for this purpose.

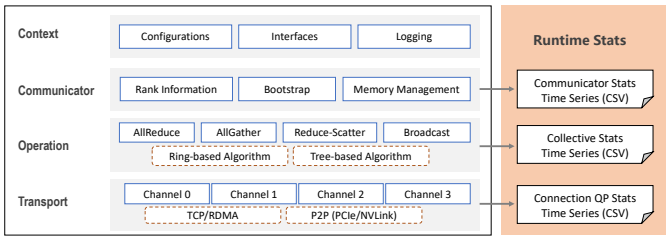



Fig. 6: The Enhancement of ACCL.

**C4D analysis.** By leveraging the gathered information from ACCL, we can build the automatic error-detection system to deal with the four common types of errors that frequently occur in our clusters, i.e., *communication hang*, *non-communication hang*, *communication slow*, and *non-communication slow*. Detecting the first two error types is relatively easy and will not be discussed in-depth here, and we will focus on the more complex task of identifying instances of the *slow* syndrome.

*Communication Slow Detection*: Before delving into the details, we will give a brief introduction to the data flow in collective communication. Considering the *allreduce* operation in data parallel phase as an example, where workers need to average gradients across all model replicas. Frameworks and CCLs will split messages into chunks and submitting them to the network sequentially. Since all workers adhere to the same data splitting rules, ensuring that message sizes remain consistent when posted to the network. Therefore, we can identify communication slowdowns at the transport layer by

monitoring and comparing the completion time of messages in different workers. An illustrative example is shown in Fig. 7. Communication delays across workers are mapped into a two-dimensional matrix, where each element represents the delay between a pair of workers, identified by their y-coordinate (source) and x-coordinate (destination). Large numbers in the matrix help identify slow connections: a single large number indicates a specific connection bottleneck, a row of large numbers suggests a problem with the source, and a column of large numbers points to an issue with the destination.

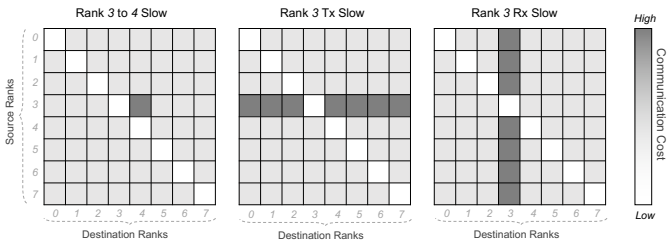



Fig. 7: The symptom for communication slow.

*Non-Communication Slow Detection*: Consider the *ring-based* algorithm in *allreduce* operation as an example. In this configuration, all participating workers are connected sequentially, forming a ring-like structure. Each worker only communicate with its immediate neighbors in the sequence, specifically the "previous rank" and the "next rank". Particularly, workers receive a data chunk from the "previous rank", perform a reduction operation with their local data chunk, and then pass the resultant data to the "next rank". In fact, due to the requirement that the receiver must first prepare the receive buffer and notify the sender before data transmission can occur, there is an implicit "receiver-driven" scheduling logic. Hence, successful communication completion in each step depends on both the sender and receiver being in a ready state. This implicit dependency is then spread to following receivers, forming a chain of pending receivers. By comparing the wait time of receivers, we can pinpoint the ranks that are experiencing non-communication slows, such as those caused by additional computation or data loading costs.

### *B. Mitigating Communication Cost*

Parallel training performance depends on the efficiency of single-node computation, data access, and collective communication. While single-node efficiency can be improved through optimizations like mixed precision [38] and transformer engine [36], and data access efficiency through caching mechanisms like Alluxio [27] , this paper focuses on collective communication efficiency, a key factor in training scalability.

If we consider network bandwidth as a resource, optimizing collective communication performance is equivalent to finding an optimal method for resource allocation. In fact, collective communication can be seen as a collection of one-to-one communications between two of the workers, which may also involve computation if a reduce operation is included. Therefore, the problem of finding an optimal resource allocation can be broken down into two issues. First, we need to minimize the resource requirements of each one-to-one communication. Second, we need to map each one-to-one communication to network resources in a way that minimizes the total communication time.

The first issue is not the focus of this paper. But for the sake of completeness, we provide a brief introduction here. Once the data volume for one-to-one communication is fixed, its consumption of network resources is directly proportional to the distance it travels through the network. To reduce the data transmission distance, we employ two optimization strategies. First, we minimize the network diameter by leveraging high-speed NVLink interconnects, which are inherently part of the network. Second, we utilize topology-aware scheduling techniques to ensure that the two ranks needing to communicate are as close as possible within the network. These optimizations mitigate the probabilities of traffic collision, and are effective for small-scale jobs.

Furthermore, we have also integrated intra-job traffic engineering techniques [13], [14] to prevent traffic collisions. However, in the case of larger AI clusters with multiple concurrent jobs, there is a lack of coordination among independent jobs. Consequently, additional optimization is required to deal with inter-job traffic collision.

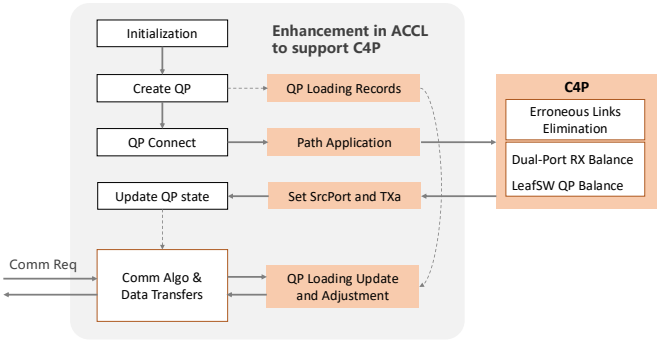



Fig. 8: The architecture of C4P.

**Our approach: C4P.** To address this problem, we introduce the C4P (C4 Performance) system, engineered to reduce unnecessary communication costs. Essentially, C4P is a cluster-scale traffic engineering technique that aims to minimize network congestion by regulating the transmission paths of each data flow within the network. This concept isn't new, but it's not commonly employed in traditional data centers, because of the massive connections in network. C4P is practical in parallel training scenarios mainly because the traffic characteristics generated by parallel training are significantly different from those of conventional cloud applications. That is, parallel training tasks involve a small number of data flows but transmit large volumes of data. This presence of a few elephant flows makes it feasible for C4P to meticulously plan the route for each data flow.

The software architecture of C4P follows that of C4D but with key differences, as shown in Fig. 8. Firstly, the C4P

master acts as a control center for multiple jobs or tenants, unlike the single-job-focused C4D master. Additionally, the C4P master records the numbers of allocated connections on each path, and allocates path for new connections considering the occupied network resources. At last, the CCL is enhanced to support issuing path allocation requests for communicating workers and set the source port accordingly. *The key functionalities of C4P can be summarized as: (1) identifying and avoiding faulty links at task start-up, (2) balancing RDMA QPs, i.e., connections, across healthy paths for load distribution, and (3) dynamically adapting QP workloads in response to network changes and traffic conflicts.*

The detailed working flow is as follows: In line with previous art [14], we utilizes path-probing to support global traffic engineering. Using this method, we can identify the source ports that will direct traffic along specific paths and verify the integrity of those paths. Initially, C4P identifies and eliminates faulty links between leaf and spine switches to establish a sub-network composed of healthy links. To do this, the C4P master performs full-mesh path probing via randomly selected servers per leaf switch, identifying and cataloging the healthy paths. Upon establishing connections, ACCL submits path allocation requests to the C4P master, which replies with the source ports of RDMA connections. The master ensures traffic from the same NIC is balanced between left and right ports by forbidding the paths from left ports to right, and vice versa. Additionally, traffic from servers under the same leaf switch is distributed over all available spine switches to prevent network congestion. The ACCL constantly evaluates message completion times on various paths and prioritizes the fastest for data transfer. This approach helps maintain low transmission delays despite path errors or traffic congestion.

## IV. Evaluations

### A. Setup

As shown in Table II, we outline the system configurations of our operational clusters, from which we have gathered the evaluation data for C4D. The assessments pertaining to C4P, on the other hand, were carried out on a subset of this cluster. To prevent any interference with other ongoing jobs in the cluster and to ensure the integrity of our evaluation results, we have allocated a specific segment of the cluster as a controlled testing environment. This testbed comprises 16 nodes, equipped with a total of 128 GPUs. And all the nodes are directly connected to 8 dedicated leaf switches, ensuring an exclusive environment for our testing activities.

The nodes within this high-performance cluster are outfitted with 8 NVIDIA H800 GPUs and 8 BlueField-3 NICs [39]. Each NIC provides two physical 200Gbps ports, which are bonded to create a single logical 400Gbps port. These NICs are integrated into a 3-Tier Clos network [7], configured in a Fat-Tree topology with a 1:1 oversubscription rate. Leveraging Broadcom's Trident4 [4] as leaf and Tomahawk4 as spine switches [3], the cluster is capable of supporting over 10,000 GPUs. In a single pod, which constitutes a two-tier subnet, it can accommodate 512 GPUs.

TABLE II: Configurations

| | |
|---|---|
| Benchmarks for C4D | GPT model, 175 billion parameters |
| Benchmarks for C4P | allreduce operations<br>GPT model, 22 billion parameters<br>Llama model, 13 billion parameters<br>GPT model, 175 billion parameters |
| Frameworks | Megatron-LM [49], DeepSpeed [45] |
| Nodes | NV H800 * 8, BlueField3 * 8 (200Gbps * 2) |
| Network | 3-Tier Clos, Fat-Tree, 1:1 oversubscription rate |

TABLE III: Error-induced Downtime Statistics

| **Error-induced Downtime in Jun, 2023** | | | |
|---|---|---|---|
| Post-Checkpoint | 7.53% | | |
| Detection | 3.41% | | |
| Diagnosis & Isolation | 19.65% | ECC/NVLink Error | 8.34% |
| | | CUDA Error | 4.19% |
| | | CCL Timeout | 3% |
| | | ACK Timeout | 1.8% |
| | | Unknown | 2.29% |
| Re-Initialization | 0.6% | | |
| Total | 31.19% | | |

| **Error-induced Downtime in Dec, 2023** | | | |
|---|---|---|---|
| Post-Checkpoint | 0.23% | | |
| Detection | 0.05% | | |
| Diagnosis & Isolation | 0.73% | ECC/NVLink Error | 0.2% |
| | | CUDA Error | 0.1% |
| | | CCL Timeout | 0.23% |
| | | ACK Timeout | 0.1% |
| | | Unknown | 0.1% |
| Re-Initialization | 0.15% | | |
| Total | 1.16% | | |

The effectiveness of C4D is evaluated with one of our real-world LLM training workloads. And the model for this job contains 175 billion parameters. Training with 2400 GPUs, it requires more than a month for a complete training cycle from scratch. In contrast, the efficiency of C4P is assessed using both collective communication benchmarks and three representative LLM training jobs. To ensure unbiased evaluation results for the communication benchmarks, we configured them to utilize a ring-based algorithm. The specific configurations of the models involved in the testing, such as parallel strategy and zero-optimizer level, will be detailed alongside the presentation of the evaluation outcomes.

### B. Results

*1) The Evaluations on C4D's Effectiveness.:* We have reviewed the historical logs of the jobs from one of our inner customers, including their start and terminated timestamps, and whether a job is succeeded or failed by some reasons. Our primary emphasis has been placed on a representative job that demands 2400 GPUs and necessitates over a month to complete. As shown in Table III, we compare the error-induced downtime of this job before (Jun, 2023) and after (Dec, 2023) the proposed fault-tolerant system was deployed. The data clearly indicates that the deployment of C4D led to a substantial decrease in downtime, with an approximately 30x reduction, from 31.19% down to just 1.16%.

The downtime in in June 2023 is broken down into four parts, according to the classification in Fig. 2. It reveals that

the majority of the downtime was due to extended periods of system diagnosis. This was largely because identifying the precise cause of job failure could take hours or even days in the absence of our diagnostic tools. The second major contributor to downtime was post-checkpoint overhead. That is because the users often scheduled infrequent checkpoints to save the model parameters, not anticipating high error rates in a large-scale training cluster. This led to a significant amount of computing activity being rendered useless between the last valid checkpoint and the moment the job failed due to an error. To address this issue and reduce the amount of wasted computing time, users have begun saving checkpoints more frequently by taking advantage of the high-performance infrastructure provided [53]. Besides the diagnosing and post-checkpoint cost, there is also 3.4% of time spent on crash detection. Typically, users are not immediately aware that their job has stalled following an error event. For instance, PyTorch jobs that might experience a hang for up to 30 minutes, until the elastic agent [42] kills the processes. This delay creates a span of idle time for computing resources, and thus constitutes a non-negligible portion of the total downtime.

In December 2023, despite system diagnosis continuing to account for the majority of downtime, its contribution had been reduced by approximately 27x compared to past performance. which is somewhat lower than the average efficiency gains observed. The deployment of C4D has markedly accelerated both error detection and the pinpointing of faulty components, cutting down the response time to mere tens of seconds. Nevertheless, additional minutes are still required by the steering service to isolate the affected nodes and restart the job, indicating room for further improvement. The second leading cause of downtime remains the post-checkpoint duration. However, with the adoption of more frequent checkpointing, users can now save checkpoints every 10 minutes. As a result, the post-checkpoint downtime has been significantly reduced by 33x. The cost associated with job re-initialization has remained constant. It has decreased from 0.6% to 0.15% of the total time, because the average error rate has decreased by 3.33x, after the most vulnerable components of the system were identified and enhanced during this six months.

Upon delving deeper into the diagnosis and isolation overhead, it has become apparent that the majority of errors are attributable to GPU defects. These include GPU ECC Errors, NVLink Errors, and CUDA Errors. In June 2023, such GPU-related issues were responsible for 12.53% of the total downtime, which constituted around 2/3 of the overall overhead. Notably, by December 2023, the incidence of these GPU-related errors had been reduced by 3.2x, while the associated time overhead saw a more dramatic decrease by 41.8x. This significant improvement can be credited to the effectiveness of C4D in handling these specific types of errors, allowing for a substantial reduction in corresponding overheads. Regarding other types of errors that are only partially manageable by C4D, there was still a noteworthy improvement. The frequency of these errors was decreased by 3.4x, and the time spent addressing them saw a reduction by 16.5x. These numbers

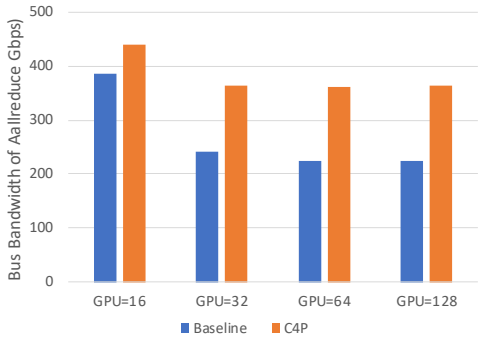
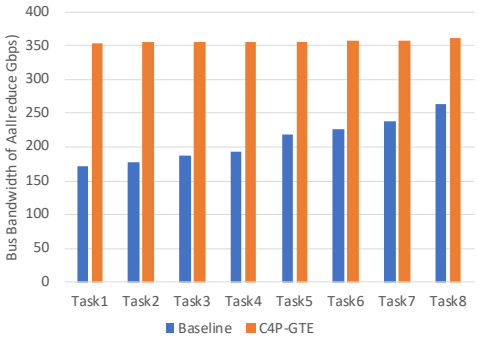



Fig. 9: Improvement from balanced traffic between bonded ports.

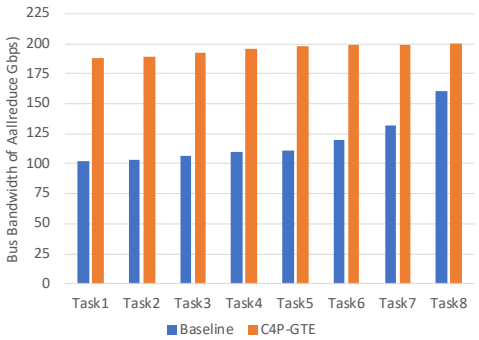
underscore the positive impact of C4D on overall system performance and reliability.

*2) The Evaluations on C4P's Effectiveness.:* **Balancing traffic between the two bonded physical ports.** As discussed in Section II-D, when two flows are dispatched from two distinct physical ports of a NIC, there is a possibility that both flows could be routed to the same physical port on the receiving end, leading to an imbalance of traffic between the two physical ports in receivers. By employing C4P, we can designate a dedicated path for the flows in each physical port, thus preventing performance degradation caused by such imbalances. Running "nccltest", the collective operation benchmarking tool supplied by NVIDIA, we are able to measure the average *busbw*, a metric that reflects the effective communication performance (higher *busbw* means lower communication delay). Fig. 9 shows the performance of a single *allreduce* operation with and without C4P enabled. Here, the results from the two-node (16-GPU) test case don't align with the effective bandwidth observed in the network. This discrepancy can be attributed to the methodology employed to calculate *busbw* within the benchmarks. We will not delve into this particular issue in this paper, and instead concentrate on other test cases to better illustrate our research outcomes. Without C4P, the effective *busbw* is lower than 240 Gbps in most test cases, which is far from the ideal bandwidth of network (around 360 Gbps). However, with C4P activated, the effective *busbw* increases close to the peak value 360 Gbps, which means 50% performance gain, showcasing the tangible benefits of C4P in optimizing network performance.

**Balancing traffic among multiple jobs.** To demonstrate the efficacy of C4P in alleviating traffic collisions across multiple concurrent jobs, we conducted an evaluation involving 8 simultaneous *allreduce* benchmarking applications. Each job was assigned two servers that are connected to distinct groups of leaf switches, ensuring that the traffic between workers traversed the spine switches. With 8 concurrent jobs in play, the network bandwidth was expected to be saturated by these parallel flows. Under these conditions, any suboptimal path selection could lead to an imbalanced traffic distribution, with some of the network links are overloaded. Such imbalances can severely impact the performance of collective communication, making the role of C4P crucial in maintaining peak communication




(a) Allreduce Performance in the network with 1:1 oversubscription.




(b) Allreduce Performance in the network with 2:1 oversubscription.

Fig. 10: Effectiveness of global traffic engineering.

performance and preventing potential bottlenecks within the network infrastructure.

Fig. 10a illustrates the performance outcomes of running concurrent *allreduce* benchmarks within a network that maintains a 1:1 oversubscription rate. The data indicates that all tasks exhibit similar levels of performance with C4P enabling global traffic engineering, closely approaching the maximum achievable throughput for a single task on this testbed. Specifically, the performance metrics for these tasks range between 353.86 Gbps and 360.57 Gbps. In contrast, when C4P is not enabled, there is a substantial performance degradation and variation among the tasks. The lowest-performing task achieves only 171.93 Gbps, while the best-performing task reaches 263.27 Gbps. On average, C4P improves the overall system throughput by 70.3%. This comparison clearly highlights the substantial improvements in bandwidth utilization and consistency enabled by C4P.

It is important to clarify that the peak throughput in this system is constrained by the NVLink fabric, causing the maximum attainable bus bandwidth at 362 Gbps. Because of the limitation of NVLink fabric, the network's capacity is underutilized, which results in an absence of queue buildup in switches and congestion-induced transmission rate reduction in senders. To evaluate the performance of C4P within a congested network environment, we intentionally reduced the number of active spine switches by half, effectively increasing the network oversubscription rate to 2:1. Under this new configuration, we replicated the previous experiments to observe how C4P handles increased traffic conditions. The outcomes of these tests are presented in Fig. 10b. A similar conclusion can be drawn from these outcomes, with the exception that the performance of concurrent tasks exhibits slight variations, when C4P is enabled. Specifically, we observe a small gap of just 11.27 Gbps between the highest and lowest throughputs. This difference is attributed to the congestion control mechanism in RDMA NICs, which dynamically adjusts the data transmission rate at the sender side in response to network conditions. As depicted in Fig. 11, each bonded port receives approximately 15,000 Congestion Notification Packets (CNPs) every second, with the count fluctuating between 12,500 and 17,500. The arrival of CNPs instigates a throttling of the data transfer rate from the senders. Fluctuations in CNP receiving frequency directly contribute to the observed variability in the flows' effective bandwidth. Despite these variations, the overall throughput is substantially improved by 65.55%, and the long-tail problem is obviously alleviated.

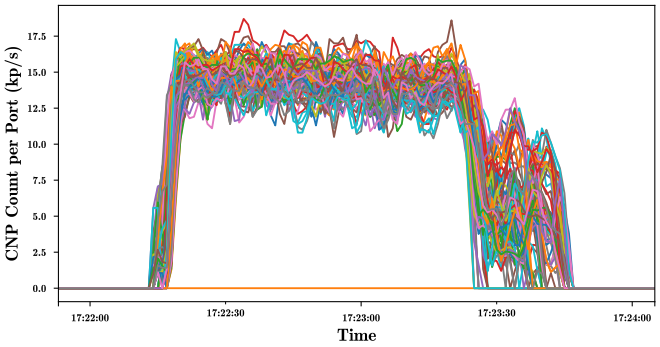
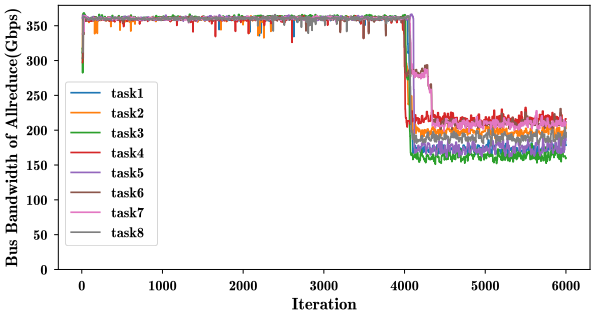
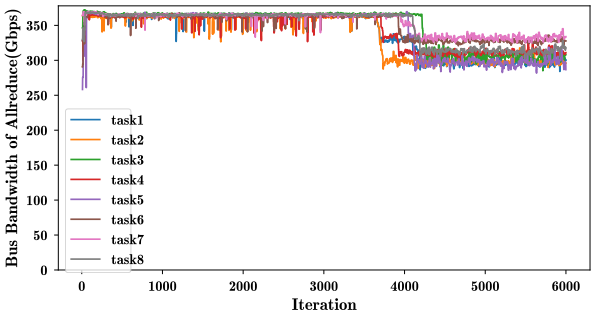



Fig. 11: The CNP count received in each bonded port.

**Tolerance to dynamic link failures.** The efficacy of global traffic engineering relies on the stability of the underlying network conditions. A down-link can necessitate the rerouting of impacted flows, which could lead to imbalanced traffic distribution within the network, and dismiss the efficacy of global traffic engineering. In this case, load balance mechanism would take into effect to adjust the load on each flow, thereby rebalancing network traffic. To assess the effectiveness of this mechanism, we replicated previous experiments in a 1:1 oversubscribed network and intentionally deactivated a link during the experiments. To obtain the tasks' instant performance, other than averaged value after the tasks are finished, we refined our testing benchmark to output results immediately upon completion of each *allreduce* operation. And the results are shown in Fig. 12. Though high-frequency timestamp recording results in some fluctuations in the data, we can still observe significant improvement in system throughput, with dynamic load balance enabled. In the tests that C4P load balancing is not activated, the bus bandwidth for the given tasks experiences considerable degradation, with values ranging between 160 Gbps and 220 Gbps and an average bandwidth of 185.76 Gbps. Conversely, when C4P load balancing is enabled, the observed bus bandwidth improves significantly,

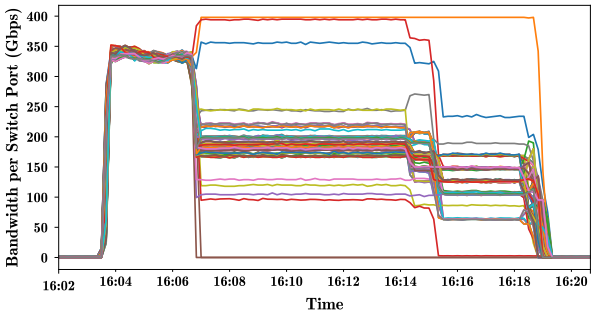



(a) Allreduce Performance with C4P static traffic engineer.

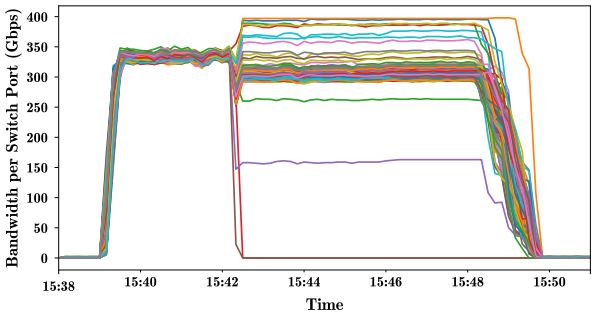



(b) Allreduce Performance with C4P dynamic load balance.

Fig. 12: Effectiveness of load balance when link failure appears.




(a) Bandwidth per switch port with C4P static traffic engineer.




(b) Bandwidth per switch port with C4P dynamic load balance.

Fig. 13: Comparison on switch port bandwidth with/without load balance.

ranging from 290 Gbps to 335 Gbps, with an average of 301.46 Gbps. This enhancement represents a substantial performance gain of 62.3% when a link error occurs during the training processes. Note that, with 1 link error among the 8 uplinks, the theoretical ideal performance would be 7/8 of the original, i.e., 315 Gbps. And the performance achieved with C4P enabled is found to be in close proximity to this ideal value.

To demonstrate how C4P load balance improves the system throughput, we have collected the statistics from the leaf switches regarding the traffic distribution across each port, as illustrated in Fig. 13. With C4P traffic engineering implemented, all upstream ports exhibit near-optimal bandwidth utilization prior to the induction of a link error. However, the emergence of the link error leads to a marked discrepancy in throughput across different ports. In the absence of C4P load balancing, only three of the ports exhibit traffic increment, indicating that the flows intended for the disabled link are being rerouted to these ports. As a result, the available bandwidth for each flow in these congested ports is reduced, and the performance of the flows in the same communication channel is also negatively affected. Consequently, a noticeable reduction in bandwidth is observed in the other ports. Conversely, C4P load balancing dynamically adjusts the distribution of network traffic, decreasing the burden on congested flows while increasing it on those underutilized paths. This strategy leads to a more balanced load across the healthy links, and results in an overall enhancement of the system's throughput.

**Performance improvement in real-life jobs.** To evaluate the efficacy of C4P in enhancing the performance of real-world

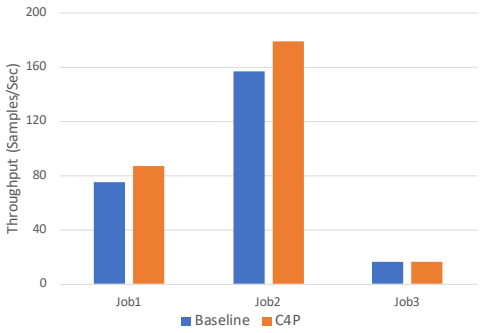



Fig. 14: Performance improvement in real-life jobs.

applications, we conducted tests using three representative training jobs: Job1 involves training a GPT model utilizing the Megatron framework, incorporating both TP and DP strategies for distributed training. This model comprises 22 billion parameters, and the TP and DP sizes are set to 8 and 16, respectively. Job2 trains a Llama model, which contains 7 billion parameters. For this task, the Deepspeed framework is employed for distributed training. The zero-optimization [45] is activated, with the training leveraging only DP. Job3 entails training another GPT model with 175 billion parameters. This model is also trained based on Megatron framework, using both TP and PP strategies. With both TP and PP sizes configured as 8, it effectively creates 2 DP groups.

The performance evaluation results are depicted in Fig. 14. As can be discerned from the figure, significant improvements in the performance of the first two jobs. Specifically, the

throughput of Job1 has increased by 15.95%, rising from 74.82 to 86.76 samples per second. Similarly, Job2 shows an enhancement of 14.1%, with throughput climbing from 156.59 to 178.65 samples per second. In contrast, Job3 does not exhibit a noticeable improvement in performance. Our analysis suggests that the discrepancy in performance gains is closely associated with the proportion of communication time within a training step. For the first two jobs, the communication overhead constitutes over 30% of each iteration's duration. However, Job3 is configured with a high Gradient Accumulation (GA) value of 16, which implies the communication cost is reduced by 16x. This setting substantially reduces the relative communication cost, which explains the minimal performance improvement observed for Job3.

## V. Related Work and Discussion

To better utilize GPU resources in LLM training, existing research makes effort in two dimensions: stability improvement and performance improvement.

**Improving stability by monitoring and fault tolerance.** Existing research on data center monitoring primarily focuses on general applications within data centers [57], [23], [25], focusing on the behavior of hardware components such as CPUs, RDMA NICs, and switches. However, when compared to conventional data center applications, the training of LLMs is characteristically on a much larger scale and involves more complex interactions between software and hardware modules. This complexity necessitates more sophisticated, high-granularity monitoring solutions [46], [40] that are integrated within the software stack.

In terms of fault tolerance, the prevalent approach is the use of in-memory checkpointing [54], [34], [31], [16]. These efforts copy the current LLM states to the host memory with high frequency such as one checkpoint per iteration. In the event of an error, once the issue has been identified, training can resume from the most recent checkpoint.

In addressing the challenge of straggler components, Jiang et al. [22] proposed an optimization solution to enhance the efficiency of large-scale AI clusters. Specifically, they identified slow components by monitoring the execution time of specific computing kernels. However, this approach is not suitable for cloud environments.

**Improving performance by better training architectures and network.** With given computation resources, the performance of LLM training can be improved by multiple-layer efforts, including (from the higher layer to the lower layer) training architectures, communication libraries, and network structure. The LLM training tries to parallelize GPU computation with communication and CPU operations [58], [46], [48], or change the behavior of LLM training to reduce the overload, leveraging the properties of specific model structures [9], [28]. Communication libraries try to better utilize the given network bandwidth, including NVLink, PCIe, and RNIC, to shorten the delay of a data transmission [21], [30], [14], [17], or design better strategies to aggregate data in collective communication [47], [8], [6]. Network structure research explore reasonable topologies to connect multiple hosts with appropriate bandwidth, providing better performance in LLM training.

Adaptive routing [10] and packet spraying [12] generally point to the same solution, which involves performing per-packet load balancing within switches based on network conditions. The decision to implement adaptive routing is entirely up to the end users, who must configure the related condition variables of CCL. However, this approach poses significant challenges, as the efficiency of adaptive routing can be compromised by the flows that are deterministic routed.

**Applicability and Limitations of C4.** C4D has broad applicability in business AI training tasks that rely on collective communication. Although we demonstrate its effectiveness using DP as an example, it is natural to wonder whether it can also be used in other scenarios, such as Pipeline Parallel (PP) [20] and Expert Parallel (EP) [26]. While C4D may not be able to detect non-communication hang or slow events within PP stages, as it relies on send/recv operations rather than collective communication, any resulting hang or slow effects would propagate to subsequent DP stages and could still be caught by C4D. In the case of EP, load imbalance among workers may occur, which can be mitigated by averaging collected data over a predefined period to smooth out random variations and highlight systemic issues. In future work, we plan to incorporate load variation into C4D to enable more precise slow detection.

The primary limitations of C4 includes: C4D's error detection capabilities rely on syndromes observed through our enhanced communication library. Consequently, C4D has limited ability to detect errors occurring during the job initialization phase, before collective communication operations are initiated. And deploying C4P requires detailed knowledge of the system configuration, including network topology, the number of switches at each layer, the number of ports per switch. Adapting C4P to different clusters necessitates retrieving this information from a background management system.

## VI. Acknowledgment

This work was partially supported by Research Grants Council of HKSAR (16213824). We thank Yu Zhou for developing the initial version of C4D.

## VII. Conclusion

Achieving optimal performance in training LLMs within large-scale AI clusters is challenging, hindered by frequent errors and traffic congestion. To address these issues, we have developed C4, a communication-centric solution designed to minimize system downtime and enhance communication efficiency. By leveraging the characteristics of distributed training, C4 can substantially reduce the system recovery cost when faced with non-correctable errors. It achieves this through rapid online fault detection, immediate isolation of anomalies, and by working in synergy with regular checkpoints and automated restart mechanisms. Additionally, C4 employs the predictable patterns of collective communication to implement

precise traffic management, which greatly alleviates network congestion. C4 can reduce error-induced overhead by roughly 30% and improve their throughput by about 15% for tested applications with moderate communication costs.

## REFERENCES


[1] M. K. Aguilera, R. Janakiraman, and L. Xu, "Using erasure codes efficiently for storage in a distributed system," in *2005 International Conference on Dependable Systems and Networks (DSN'05)*. IEEE, 2005, pp. 336–345.

[2] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang, B. Hui, L. Ji, M. Li, J. Lin, R. Lin, D. Liu, G. Liu, C. Lu, K. Lu, J. Ma, R. Men, X. Ren, X. Ren, C. Tan, S. Tan, J. Tu, P. Wang, S. Wang, W. Wang, S. Wu, B. Xu, J. Xu, A. Yang, H. Yang, J. Yang, S. Yang, Y. Yao, B. Yu, H. Yuan, Z. Yuan, J. Zhang, X. Zhang, Y. Zhang, Z. Zhang, C. Zhou, J. Zhou, X. Zhou, and T. Zhu, "Qwen technical report," 2023.

[3] BROADCOM, "Tomahawk4 / bcm56990 series," https://www.broadcom.com/products/ethernet-connectivity/switching/strataxgs/bcm56990-series.

[4] ——, "Trident4 / bcm56880 series," https://www.broadcom.com/products/ethernet-connectivity/switching/strataxgs/bcm56880-series.

[5] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[6] Z. Cai, Z. Liu, S. Maleki, M. Musuvathi, T. Mytkowicz, J. Nelson, and O. Saarikivi, "Synthesizing optimal collective algorithms," in *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2021, pp. 62–75.

[7] C. Clos, "A study of non-blocking switching networks," *The Bell System Technical Journal*, vol. 32, no. 2, pp. 406–424, 1953.

[8] M. Cowan, S. Maleki, M. Musuvathi, O. Saarikivi, and Y. Xiong, "Msccclang: Microsoft collective communication language," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, pp. 502–514.

[9] W. Cui, Z. Han, L. Ouyang, Y. Wang, N. Zheng, L. Ma, Y. Yang, F. Yang, J. Xue, L. Qiu *et al.*, "Optimizing dynamic neural networks with brainstorm," in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, 2023, pp. 797–815.

[10] Dally and Seitz, "Deadlock-free message routing in multiprocessor interconnection networks," *IEEE Transactions on Computers*, vol. C-36, no. 5, pp. 547–553, 1987.

[11] A. G. Dimakis, V. Prabhakaran, and K. Ramchandran, "Decentralized erasure codes for distributed networked storage," *IEEE Transactions on Information Theory*, vol. 52, no. 6, pp. 2809–2816, 2006.

[12] A. Dixit, P. Prakash, Y. C. Hu, and R. R. Kompella, "On the impact of packet spraying in data center networks," in *2013 Proceedings IEEE INFOCOM*, 2013, pp. 2130–2138.

[13] J. Dong, Z. Cao, T. Zhang, J. Ye, S. Wang, F. Feng, L. Zhao, X. Liu, L. Song, L. Peng, Y. Guo, X. Jiang, L. Tang, Y. Du, Y. Zhang, P. Pan, and Y. Xie, "Eflops: Algorithm and system co-design for a high performance distributed training platform," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2020, pp. 610–622.

[14] J. Dong, S. Wang, F. Feng, Z. Cao, H. Pan, L. Tang, P. Li, H. Li, Q. Ran, Y. Guo *et al.*, "Accl: architecting highly scalable distributed training systems with highly efficient collective communication library," *IEEE Micro*, 2021.

[15] Z. Du, Y. Qian, X. Liu, M. Ding, J. Qiu, Z. Yang, and J. Tang, "Glm: General language model pretraining with autoregressive blank infilling," 2022.

[16] A. Eisenman, K. K. Matam, S. Ingram, D. Mudigere, R. Krishnamoorthi, K. Nair, M. Smelyanskiy, and M. Annavaram, "{Check-N-Run}: A checkpointing system for training deep learning recommendation models," in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, 2022, pp. 929–943.

[17] Facebook, "Gloo," 2023, https://github.com/facebookincubator/gloo.

[18] H. Han, S. Shakkottai, C. V. Hollot, R. Srikant, and D. Towsley, "Multipath tcp: A joint congestion control and routing scheme to exploit path diversity in the internet," *IEEE/ACM Transactions on Networking*, vol. 14, no. 6, pp. 1260–1271, 2006.

[19] C. Hopps, "Rfc2992: Analysis of an equal-cost multi-path algorithm," 2000.

[20] Y. Huang, Y. Cheng, A. Bapna, O. Firat, M. X. Chen, D. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, "Gpipe: Efficient training of giant neural networks using pipeline parallelism," 2019. [Online]. Available: https://arxiv.org/abs/1811.06965

[21] S. Jeaugey, "Nccl 2.0," in *GPU Technology Conference (GTC)*, vol. 2, 2017.

[22] Z. Jiang, H. Lin, Y. Zhong, Q. Huang, Y. Chen, Z. Zhang, Y. Peng, X. Li, C. Xie, S. Nong, Y. Jia, S. He, H. Chen, Z. Bai, Q. Hou, S. Yan, D. Zhou, Y. Sheng, Z. Jiang, H. Xu, H. Wei, Z. Zhang, P. Nie, L. Zou, S. Zhao, L. Xiang, Z. Liu, Z. Li, X. Jia, J. Ye, X. Jin, and X. Liu, "Megascale: Scaling large language model training to more than 10,000 gpus," 2024. [Online]. Available: https://arxiv.org/abs/2402.15627

[23] R. Joshi, T. Qu, M. C. Chan, B. Leong, and B. T. Loo, "Burstradar: Practical real-time microburst monitoring for datacenter networks," in *Proceedings of the 9th Asia-Pacific Workshop on Systems*, 2018, pp. 1–8.

[24] R. Koo and S. Toueg, "Checkpointing and rollback-recovery for distributed systems," *IEEE Transactions on software Engineering*, no. 1, pp. 23–31, 1987.

[25] M. Kutare, G. Eisenhauer, C. Wang, K. Schwan, V. Talwar, and M. Wolf, "Monalytics: online monitoring and analytics for managing large scale data centers," in *Proceedings of the 7th international conference on Autonomic computing*, 2010, pp. 141–150.

[26] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, "Gshard: Scaling giant models with conditional computation and automatic sharding," 2020. [Online]. Available: https://arxiv.org/abs/2006.16668

[27] H. Li, "Alluxio: A virtual distributed file system," Ph.D. dissertation, EECS Department, University of California, Berkeley, May 2018. [Online]. Available: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2018/EECS-2018-29.html

[28] J. Liu, J. H. Wang, and Y. Jiang, "Janus: A unified distributed training framework for sparse mixture-of-experts models," in *Proceedings of the ACM SIGCOMM 2023 Conference*, 2023, pp. 486–498.

[29] Y. Lu, G. Chen, B. Li, K. Tan, Y. Xiong, P. Cheng, J. Zhang, E. Chen, and T. Moscibroda, "Multi-Path transport for RDMA in datacenters," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. Renton, WA: USENIX Association, Apr. 2018, pp. 357–371. [Online]. Available: https://www.usenix.org/conference/nsdi18/presentation/lu

[30] Microsoft, "Nsccl," 2023, https://github.com/microsoft/msccl.

[31] J. Mohan, A. Phanishayee, and V. Chidambaram, "{CheckFreq}: Frequent,{Fine-Grained}{DNN} checkpointing," in *19th USENIX Conference on File and Storage Technologies (FAST 21)*, 2021, pp. 203–216.

[32] S. S. Mukherjee, M. Kontz, and S. K. Reinhardt, "Detailed design and evaluation of redundant multithreading alternatives," *ACM SIGARCH Computer Architecture News*, vol. 30, no. 2, pp. 99–110, 2002.

[33] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. A. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia, "Efficient large-scale language model training on gpu clusters using megatron-lm," 2021. [Online]. Available: https://arxiv.org/abs/2104.04473

[34] B. Nicolae, J. Li, J. M. Wozniak, G. Bosilca, M. Dorier, and F. Cappello, "Deepfreeze: Towards scalable asynchronous checkpointing of deep learning models," in *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*. IEEE, 2020, pp. 172–181.

[35] NVIDIA, "Nvidia a100 tensor core gpu," https://www.nvidia.com/en-us/data-center/a100/.

[36] ——, "Nvidia bluefield-3 dpu," https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/documents/datasheet-nvidia-bluefield-3-dpu.pdf.

[37] ——, "Nvidia h100 tensor core gpu," https://www.nvidia.com/en-us/data-center/h100/.

[38] ——, "Train with mixed precision," https://docs.nvidia.com/deeplearning/performance/mixed-precision-training/index.html.

[39] ——, "Transformer engine documentation," https://docs.nvidia.com/deeplearning/transformer-engine/user-guide/index.html.

[40] Nvidia, "Nvidia nsight systems," 2023, https://developer.nvidia.com/nsight-systems.

[41] S. Poledna, *Fault-tolerant real-time systems: The problem of replica determinism*. Springer Science & Business Media, 2007, vol. 345.
[42] Pytorch, "Elastic agent," https://pytorch.org/docs/stable/elastic/agent.html.
[43] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever *et al.*, "Improving language understanding by generative pre-training," 2018.
[44] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
[45] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "Zero: Memory optimizations toward training trillion parameter models," 2020.
[46] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, "Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020, pp. 3505–3506.
[47] A. Shah, V. Chidambaram, M. Cowan, S. Maleki, M. Musuvathi, T. Mytkowicz, J. Nelson, O. Saarikivi, and R. Singh, "{TACCL}: Guiding collective algorithm synthesis using communication sketches," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 593–612.
[48] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," *arXiv preprint arXiv:1909.08053*, 2019.
[49] ——, "Megatron-lm: Training multi-billion parameter language models using model parallelism," 2020.
[50] J. C. Smolens, B. T. Gold, B. Falsafi, and J. C. Hoe, "Reunion: Complexity-effective multicore redundancy," in *2006 39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'06)*, 2006, pp. 223–234.
[51] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," 2023.
[52] L. G. Valiant, "A bridging model for parallel computation," *Commun. ACM*, vol. 33, no. 8, p. 103–111, aug 1990. [Online]. Available: https://doi.org/10.1145/79173.79181
[53] Z. Wang, Z. Jia, S. Zheng, Z. Zhang, X. Fu, T. E. Ng, and Y. Wang, "Gemini: Fast failure recovery in distributed training with in-memory checkpoints," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 364–381.
[54] ——, "Gemini: Fast failure recovery in distributed training with in-memory checkpoints," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 364–381.
[55] H. Weatherspoon and J. D. Kubiatowicz, "Erasure coding vs. replication: A quantitative comparison," in *International Workshop on Peer-to-Peer Systems*. Springer, 2002, pp. 328–337.
[56] M. Xia, M. Saxena, M. Blaum, and D. A. Pease, "A tale of two erasure codes in HDFS," in *13th USENIX Conference on File and Storage Technologies (FAST 15)*. Santa Clara, CA: USENIX Association, Feb. 2015, pp. 213–226. [Online]. Available: https://www.usenix.org/conference/fast15/technical-sessions/presentation/xia
[57] C. Yu, C. Lumezanu, A. Sharma, Q. Xu, G. Jiang, and H. V. Madhyastha, "Software-defined latency monitoring in data center networks," in *Passive and Active Measurement: 16th International Conference, PAM 2015, New York, NY, USA, March 19-20, 2015, Proceedings 16*. Springer, 2015, pp. 360–372.
[58] C. Zhang, L. Ma, J. Xue, Y. Shi, Z. Miao, F. Yang, J. Zhai, Z. Yang, and M. Yang, "Cocktailer: Analyzing and optimizing dynamic control flow in deep learning," in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, 2023, pp. 681–699.